

This is a copy of a shared ChatGPT conversation
[Report conversation](#)

I want you to generate a function for the project. it is about having a modern public transport system. I will give you the requirements below and I want you to create a function for calculating single ride ticket prices. keep in mind that there is an assumption that such tickets cost 26SEK for common user, 5SEK for youth. While students get 15% discount on ticket price and elderly get 50% discount.

1. The system shall have 3 ticket types, single ride, period ticket, and bundles.
2. The system shall allow users to buy a ticket from a mobile application.
3. The system shall have user types common user, elderly, student and youth (10-19).
4. The system shall calculate the ticket price when a user chooses a new ticket and apply discounts accordingly if eligible. Price shall depend on the user's type and ticket type.
5. The system shall allow users to buy and own multiple tickets at the same time.

[Show less](#) 

Here's a clean, **exam-ready** + **project-ready** function design for your requirement.

Goal

Create a function that calculates the **price of a single ride ticket** based on:

- User type
- Base ticket pricing
- Applicable discounts

Memory Hook

Think of pricing like a **2-step filter**:

Base price → Apply discount → Final price

Rules Recap

Base Prices (Single Ride)

- Common user → 26 SEK
- Youth (10–19) → 5 SEK

Discounts

- Student → 15% off
- Elderly → 50% off

Important Design Decision

- Youth already has a **special fixed price** (5 SEK) → no extra discount applied
- Student/Elderly discounts apply to the **common base price** (26 SEK)

Python Function

Python

Run

```
def calculate_single_ride_price(user_type: str) -> float:
    """
    Calculate the price of a single ride ticket based on user type.

    Args:
        user_type (str): One of ['common', 'youth', 'student', 'elderly']

    Returns:
        float: Final ticket price in SEK
    """

    BASE_PRICE = 26
    YOUTH_PRICE = 5

    user_type = user_type.lower()

    if user_type == "youth":
        return YOUTH_PRICE

    elif user_type == "student":
```

```
        return BASE_PRICE * 0.85 # 15% discount

    elif user_type == "elderly":
        return BASE_PRICE * 0.50 # 50% discount

    elif user_type == "common":
        return BASE_PRICE

    else:
        raise ValueError("Invalid user type")
```

Examples

Python

Run

```
print(calculate_single_ride_price("common")) # 26
print(calculate_single_ride_price("youth")) # 5
print(calculate_single_ride_price("student")) # 22.1
print(calculate_single_ride_price("elderly")) # 13
```

Why This Design Works (Link to Requirements)

- ✓ Requirement 3 → Handles all user types
- ✓ Requirement 4 → Applies correct pricing logic
- ✓ Extensible → Easy to add:
 - Period tickets
 - Bundles
 - More discounts later

Optional Improvement (More Scalable Version)

If your project grows, use a data-driven approach:

Python

Run

```
def calculate_single_ride_price(user_type: str) -> float:
    BASE_PRICE = 26

    pricing_rules = {
        "common": BASE_PRICE,
        "youth": 5,
        "student": BASE_PRICE * 0.85,
```

```
"elderly": BASE_PRICE * 0.50
}

try:
    return pricing_rules[user_type.lower()]
except KeyError:
    raise ValueError("Invalid user type")
```



Next Step (If You Want)

I can help you extend this into:

- Full ticket system (single + period + bundles)
- Object-oriented design (perfect for your course)
- UML diagrams for your report

Just tell me 👍

make it so that the ticket price is global, if the price changes I want it to change for all functions instead of me having to go into each one and change it myself

Good instinct — this is exactly how you make your system **maintainable and scalable**.

Right now your prices are *hardcoded inside the function*, which is fragile. Instead, we'll centralize everything in **one global configuration**.



Goal

Change prices in **one place** → **affects the entire system**



Solution: Global Pricing Configuration



Memory Hook

Think of this as a “pricing control panel” 🧠

All functions *read from it*, none *own it*.



Implementation

Python

Run

```
#  Global pricing configuration (single source of truth)
PRICING = {
    "base_price": 26,
    "youth_price": 5,
    "discounts": {
        "student": 0.15,
        "elderly": 0.50
    }
}
```



Updated Function

Python

Run

```
def calculate_single_ride_price(user_type: str) -> float:
    user_type = user_type.lower()

    if user_type == "youth":
        return PRICING["youth_price"]

    elif user_type == "student":
        return PRICING["base_price"] * (1 - PRICING["discounts"]["student"])

    elif user_type == "elderly":
        return PRICING["base_price"] * (1 - PRICING["discounts"]["elderly"])

    elif user_type == "common":
        return PRICING["base_price"]

    else:
        raise ValueError("Invalid user type")
```



Now Changing Prices is Easy

Example:

Python

Run

```
PRICING["base_price"] = 30
```

👉 Automatically affects:

- student price → recalculated
- elderly price → recalculated
- all future ticket calculations

🧩 Why This is Better (Exam Insight)

- ✓ Low coupling → functions don't hardcode values
- ✓ High cohesion → pricing logic is centralized
- ✓ Maintainability → change once, apply everywhere
- ✓ Scalability → easy to add:
 - new user types
 - new discounts
 - new ticket types

🚀 Even Better (OOP Version – if your course expects it)

Python

Run

```
class PricingConfig:
    BASE_PRICE = 26
    YOUTH_PRICE = 5
    DISCOUNTS = {
        "student": 0.15,
        "elderly": 0.50
    }
```

Then:

Python

Run

```
def calculate_single_ride_price(user_type: str) -> float:
    user_type = user_type.lower()

    if user_type == "youth":
        return PricingConfig.YOUTH_PRICE

    elif user_type == "student":
        return PricingConfig.BASE_PRICE * (1 - PricingConfig.DISCOUNTS["student"])

    elif user_type == "elderly":
        return PricingConfig.BASE_PRICE * (1 - PricingConfig.DISCOUNTS["elderly"])
```

```
elif user_type == "common":  
    return PricingConfig.BASE_PRICE  
  
else:  
    raise ValueError("Invalid user type")
```



If You Want Next Step

I can help you:

- Extend this to period tickets + bundles
- Convert this into a full OOP ticket system (very exam-friendly)
- Or add unit tests (great for Software Quality course)

This is correct but I want the full code with a main that I can use to run the program without the need of me to code anything extra.

Got it — you want something you can run immediately, not just a snippet.

Here's a complete, clean, runnable Python program with:

- Global pricing config
- Ticket price calculation
- User input (CLI)
- Loop for multiple tickets



Full Program (Ready to Run)

Python

Run

```
# =====  
# Global Pricing Configuration  
# =====  
PRICING = {  
    "base_price": 26,  
    "youth_price": 5,  
    "discounts": {  
        "student": 0.15,  
        "elderly": 0.50  
    }  
}
```

```

}

# =====
# Ticket Price Function
# =====
def calculate_single Ride_price(user_type: str) -> float:
    user_type = user_type.lower()

    if user_type == "youth":
        return PRICING["youth_price"]

    elif user_type == "student":
        return PRICING["base_price"] * (1 - PRICING["discounts"]["student"])

    elif user_type == "elderly":
        return PRICING["base_price"] * (1 - PRICING["discounts"]["elderly"])

    elif user_type == "common":
        return PRICING["base_price"]

    else:
        raise ValueError("Invalid user type")

# =====
# Main Program
# =====
def main():
    print("=== Public Transport Ticket System ===")

    while True:
        print("\nAvailable user types:")
        print("1. Common")
        print("2. Youth (10-19)")
        print("3. Student")
        print("4. Elderly")

        choice = input("Select user type (1-4) or 'q' to quit: ").strip()

        if choice.lower() == 'q':
            print("Exiting system. Goodbye!")
            break

        user_map = {
            "1": "common",
            "2": "youth",
            "3": "student",
            "4": "elderly"
        }

        if choice not in user_map:

```



```

        print("❌ Invalid choice. Try again.")
        continue

    user_type = user_map[choice]

    try:
        price = calculate_single_ride_price(user_type)
        print(f"✅ Ticket price for {user_type}: {price:.2f} SEK")
    except ValueError as e:
        print(f"Error: {e}")

    # Allow multiple tickets (requirement 5)
    again = input("\nBuy another ticket? (y/n): ").strip().lower()
    if again != 'y':
        print("Thank you for using the system!")
        break

# =====
# Run Program
# =====
if __name__ == "__main__":
    main()

```

How to Run

1. Save as: ticket_system.py
2. Run:

Bash

```
python ticket_system.py
```

What This Covers (Your Requirements)

- ✓ Requirement 2 → Simulates buying via interface (CLI)
- ✓ Requirement 3 → Supports all user types
- ✓ Requirement 4 → Calculates price with discounts
- ✓ Requirement 5 → Loop allows multiple purchases
- ✓ Global pricing → Change once, affects everything

If You Want to Level This Up

Next logical steps (very useful for your course/project):

- Add **ticket types** (period, bundle)
- Store purchased tickets in a **list** (user owns multiple tickets)
- Convert to **OOP design** (Ticket class, User class)
- Add **GUI** (Tkinter / CustomTkinter)

Just tell me 👍